

MASTER DI I LIVELLO
CYBERSECURITY

UNIVERSITÀ DEGLI STUDI DI TORINO



Cod. Corso A757- 12-2024-0
ID Attività 2584660

Insegnamento: Oracles

Prof. Silvio Meneguzzo

MODULE

Blockchain Oracles

Bridging smart contracts to the real world: the oracle problem, design patterns, data authentication, computation & decentralized oracles, and cross-chain messaging

Blockchain and Distributed Ledger Technologies

Bachelor Degree in Computer Science | Academic Year 2025-2026

Structured on Ch. 11 of “Mastering Ethereum” (2nd ed.), Antonopoulos, Wood et al. — CC BY-NC-ND

Outline

I	The oracle problem	Why determinism forbids on-chain randomness & external data; the trust model
II	Use cases & taxonomy	What oracles do, the data they provide, attestations & subjective truth
III	Design patterns	Immediate-read, publish-subscribe, request-response
IV	Trust & authentication	Authenticity proofs, TEEs, computation oracles & ZK coprocessors
V	Decentralized oracles	Single points of failure, Chainlink DON, SchellingCoin / Schelling point
VI	Cross-chain messaging	LayerZero, Wormhole, Chainlink CCIP, CCTP, IBC / Polkadot
VII	Practice & references	Consuming a price feed in Solidity; best practices; references

PART I

The oracle problem

Why a deterministic blockchain cannot see the outside world on its own

What is a blockchain oracle?

An **oracle** is a system that answers questions external to the blockchain — feeding off-chain (“real-world”) data into smart contracts. The name comes from Greek mythology: a person in communication with the gods who could reveal what was otherwise unknowable.

A bridge

Oracles connect the deterministic on-chain world to off-chain data sources, other systems, and other blockchains.

Ideally trustless

A good oracle should not have to be trusted — it operates on decentralized, verifiable principles instead of a single authority.

Data, not just prices

Sports results, the gold price, weather, randomness, IDs, flight stats — anything a contract needs but cannot compute itself.

A new attack surface

By importing outside data, oracles also import outside RISK into Ethereum's security model. Trust must be designed carefully.

Why the EVM is “blind”: determinism & consensus

Every node must reach the SAME state from the same inputs, or consensus collapses. EVM execution is therefore strictly deterministic and depends only on the shared Ethereum state and signed transactions. Two consequences follow:

1 • No intrinsic randomness

If a `RANDOM()` opcode existed, node A might store 3 and node B store 7 for the same contract — they could never agree on the resulting state. So the EVM has no true randomness.

2 • External data only via transactions

Extrinsic information can enter only as the data payload of a signed transaction — and that payload comes from an unverifiable source, so it cannot simply be trusted.

Pseudorandomness (hashes) is deterministic — usable, but NOT safe alone: e.g. a block proposer could pick which coin-flip transactions to include and only keep their wins.

The “oracle problem” and the trust model

Putting data into a transaction does not make it true — **it just defers the trust problem to whoever supplied the data**. This is the oracle problem: a trustless chain is only as trustworthy as the (often trusted) data it consumes.

⚠ Worked example — the “smart will”

A contract distributes an estate when an oracle reports the owner has died. If the inheritance is large, the incentive to HACK the oracle — or bribe its human operator — and trigger an early payout is enormous. The contract is only as secure as the weakest oracle feeding it.

Takeaway: always reason explicitly about the oracle's trust assumptions. If you assume the oracle is reliable, your contract is vulnerable to every way that assumption can fail.

PART II

Use cases & taxonomy

What oracles do, the data they carry, and the limits of “truth”

Three functions every oracle performs

1 • Collect

Obtain data from an off-chain source (an API, a sensor, a database, a human attester, another chain).

2 • Transfer on-chain

Relay the data to the chain inside a SIGNED message / transaction, attesting to it.

3 • Make available

Store or expose the data so contracts can read it via a retrieve() call — and clients can read it directly.

Once on-chain, the data is reusable: other contracts call the oracle contract, and off-chain apps read it without paying gas.

A catalog of oracle use cases

Finance & markets

FX & crypto/fiat exchange rates · capital-markets pricing · interest-rate benchmarks for derivatives · ETH gas-price feeds

Insurance & risk

Parametric triggers (earthquake/Richter for cat-bonds) · weather data · damage verification

Identity & static data

Self-sovereign IDs · academic certificates · country/currency codes · security identifiers

Randomness

Entropy from physical (quantum/thermal) sources — fair lottery winner selection, gaming

Events & prediction

Sports results · political events · prediction-market & fantasy-sports resolution

Physical & cross-chain

Geolocation / supply-chain tracking · IoT sensors · flight statistics · events on OTHER blockchains

Attestations & the limits of “trustless”

Some data has no independently verifiable, objective truth — it is true only because an authority says so. These sources are inherently TRUSTED, not trustless.

Subjective-truth sources

A university issuing a degree, or a government issuing an ID. Truth = appeal to the authority of the issuer; it cannot be checked against an objective reality.

Delivered as attestations

Passports, diplomas, memberships, records of achievement. We still call them oracles: they bridge external facts to contracts.

Identity & reputation

Attestations are expected to be central to blockchain's future — verifying identity and reputation on-chain.

Privacy matters

Issuers store COMMITMENTS, not raw data (next section) — e.g. a hash or a Merkle root — so private details never go on-chain.

PART III

Oracle design patterns

Three ways to deliver data: immediate-read, publish-subscribe, request-response

Pattern 1 • Immediate-read

The simplest oracle: data needed for an immediate, one-off decision — “What is the address for ethereumbook.info?”, “Is this person over 18?”. Queried just-in-time, perhaps never again.

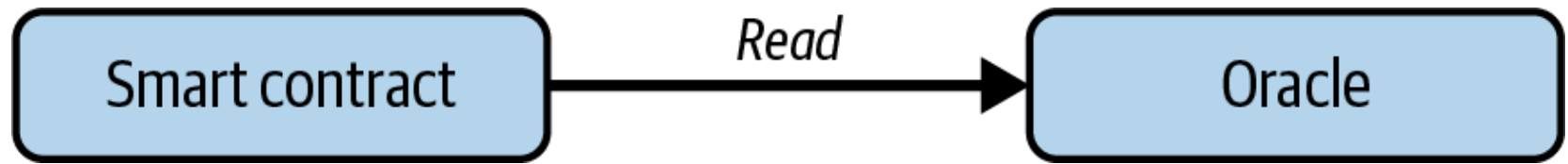


Figure 11-1 — immediate-read oracle. Source: Mastering Ethereum 2nd ed. (CC BY-NC-ND).

Stored once, read on demand

Data lives in the oracle's storage; any contract can look it up with a call, and off-chain apps can read it directly — no transaction, no gas.

Store a commitment, not raw data

For privacy/efficiency, store a HASH (or a Merkle root with salts), e.g. degree certificates or citizen IDs — never the full record.

Pattern 2 • Publish-subscribe

For data that changes regularly (price feeds, weather, statistics). The oracle is updated and flags that new data is available; “subscribers” poll it on-chain or watch via event logs off-chain — much like RSS / WebSub.

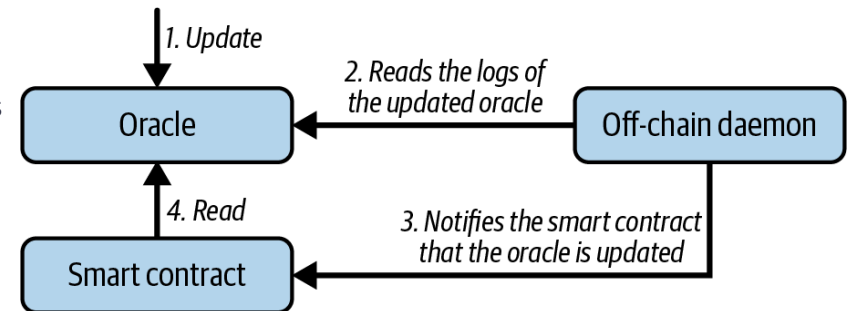


Figure 11-2 — publish-subscribe oracle. Source: Mastering Ethereum 2nd ed. (CC BY-NC-ND).

Polling is cheap here

Unlike web servers, Ethereum clients already track all state changes — polling synced contract storage is a local call. Event logs make it feel like “push”.

Timestamped updates

The oracle can store the time of each update; a reading contract sees the data's age and decides whether to trust it — no off-chain daemon needed.

Pattern 3 • Request-response

For datasets too large to store on-chain, where users need only a small slice at a time (and for data-provider businesses). An ASYNCHRONOUS client-server flow built from on-chain contracts + off-chain infrastructure.

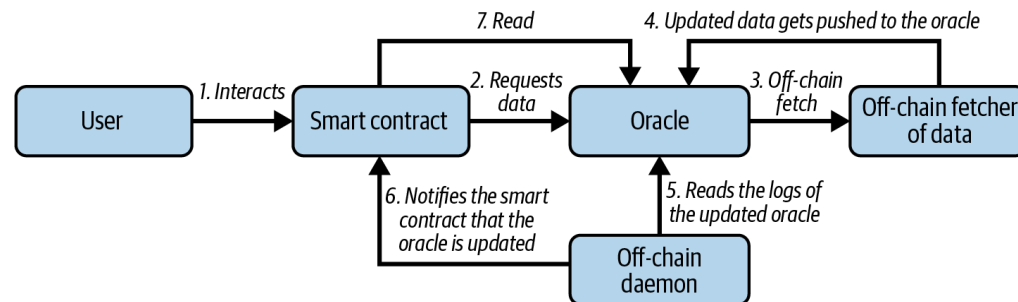


Figure 11-3 — request-response oracle. Source: Mastering Ethereum 2nd ed. (CC BY-NC-ND).

Asynchrony matters: the answer arrives in a LATER transaction (via callback), so the requesting DApp must be designed to handle delayed responses — it cannot expect data in the same call.

Request-response • the seven steps

- 1 Receive a query from a DApp (an EOA triggers the oracle contract's request function, with arguments + callback + scheduling).
- 2 Parse the query (the request is emitted as an EVM event / state change the off-chain node observes).
- 3 Check payment and data-access permissions (request fee, gas for the callback, access rights).
- 4 Retrieve the data from the off-chain source (encrypting it if necessary).
- 5 Sign the transaction(s) with the data included — attesting to its validity at that time.
- 6 Broadcast the transaction(s) back to the DApp, directly or via the oracle contract.
- 7 Schedule any further transactions (e.g. recurring end-of-day price updates, notifications).

Oracles can be human, software, or hardware (e.g. an IoT sensor answering directly). An EOA can also respond without any oracle contract.

Choosing a pattern

Pattern	Best when...	Data size / freshness	Example
Immediate-read	A one-off, just-in-time lookup	Small, rarely changing	Age check, certificate, ID
Publish-subscribe	Data changes regularly; many readers	Small, frequently updated	Price feed, weather, rates
Request-response	Dataset too big to store; need a slice	Huge, on-demand	Custom queries, data business

Rule of thumb: a value that changes infrequently (e.g. an interest rate) suits publish-subscribe; forcing a daily request-response wastes Ethereum's limited bandwidth.

PART IV

Trust & authentication

Proving the data was not tampered with — and offloading computation

Data authentication

Even if the SOURCE is authoritative, the oracle and transport may be operated by others — data could be tampered with in transit. Two complementary techniques attest to integrity:

Authenticity proofs

Cryptographic guarantees that data is unchanged from source to chain. A signature or zero-knowledge attestation is verified ON-CHAIN, shifting trust from the transport to a verifiable attester (provider, third party, or node network).

Trusted Execution Environments (TEEs)

Hardware enclaves isolate code & data from the host. The CPU produces a signed ATTESTATION that a specific (hash-identified) program ran untampered — and can keep sensitive inputs encrypted/private.

⚠ TEEs are relatively new and under-tested — expect undiscovered vulnerabilities and future exploits. Don't treat them as infallible.

Authenticity proof in action • Chainlink VRF

Chainlink VRF (Verifiable Random Function) is a concrete authenticity proof — it solves the “no randomness” problem and proves the result was not manipulated:

- 1 A contract requests a random number, supplying a SEED/hint the oracle cannot predict.
- 2 Each oracle uses its PRIVATE KEY to generate the random value off-chain, plus a cryptographic proof.
- 3 The value AND its proof are published on-chain.
- 4 The contract verifies the proof using the oracle's PUBLIC KEY + the original hint — before anyone can consume the value.
- 5 Tampering invalidates the proof; an unresponsive/compromised node is recorded on-chain and excluded.

Computation oracles & ZK coprocessors

Oracles need not only relay data — they can perform heavy COMPUTATION off-chain that would be infeasible or too costly under Ethereum's block gas limit (e.g. a complex regression to estimate a bond's yield), then return the result.

ZK coprocessors

Lagrange and Brevis let contracts run intensive tasks / historical-data queries off-chain with ON-CHAIN verification via zero-knowledge proofs.

Optimistic model (Brevis)

Results are posted “optimistically” (assumed correct) with a CHALLENGE window. A dispute forces the proposer to produce a full ZK proof; false results are slashed, honest challengers rewarded.

Benefit: dramatically fewer ZK proofs need to be generated in the common (unchallenged) case → much lower cost, while preserving verifiability.

PART V

Decentralized oracles

Removing the single point of failure

Why decentralize? The Chainlink DON

A centralized oracle is a **single point of failure**. Decentralized oracle networks (DONs) spread trust across many independent providers with on-chain aggregation. Chainlink's design uses three contracts + an off-chain registry:

Reputation contract

Tracks each data provider's historical performance; scores populate the off-chain registry.

Order-matching contract

Takes a request, selects oracle bids using reputation, and finalizes an SLA (query params + number of oracles). The buyer never deals with oracles individually.

Aggregation contract

Collects responses (via a commit-reveal scheme) from multiple oracles, computes the collective result, and feeds outcomes back to reputation.

Economic security

Nodes are incentivized and often bonded — a single bad actor can't easily corrupt a multi-operator feed.

Aggregation & the SchellingCoin protocol

The hard part is the AGGREGATION function. Chainlink computes a weighted response with a validity score per oracle — but flagging “outliers” as wrong risks penalizing CORRECT answers that simply differ from the average. Hence custom aggregation contracts are allowed.

SchellingCoin / the Schelling point

- Many participants independently report a value; the **MEDIAN** is taken as the “correct” answer.
- Each reporter posts a DEPOSIT, redistributed in favour of values closest to the median — so honesty (matching others) is the profitable strategy.
- A **Schelling point** is the natural, obvious value rational players coordinate on without communicating — expected to sit near the true value.

PART VI

Cross-chain messaging protocols

From ingesting external data to connecting whole ecosystems

Oracles vs cross-chain protocols

Different chains have their own governance, consensus and token standards. Cross-chain protocols are specialized “communication layers” that connect entire ecosystems — extending the oracle idea from data ingestion to inter-chain messaging.

An oracle

Brings EXTERNAL information INTO one blockchain (off-chain → chain).

A cross-chain protocol

Transfers information and assets BETWEEN blockchains (chain ↔ chain), unlocking shared liquidity, services and data.

Common need: securely attest that “event X happened on chain A” so chain B can act on it — token bridges, cross-chain governance, NFT transfers.

LayerZero — lightweight message passing

LayerZero focuses on transporting and validating messages across chains using two independent off-chain entities:

Oracle

Independently queries the source transaction's proof / block header.

Relayer

Independently passes the transaction proof itself.

If the Oracle and Relayer **AGREE**, LayerZero's destination-chain contracts accept the message. The set of Oracles/Relayers is user-configurable → trust is decentralized, with no single bridging provider.

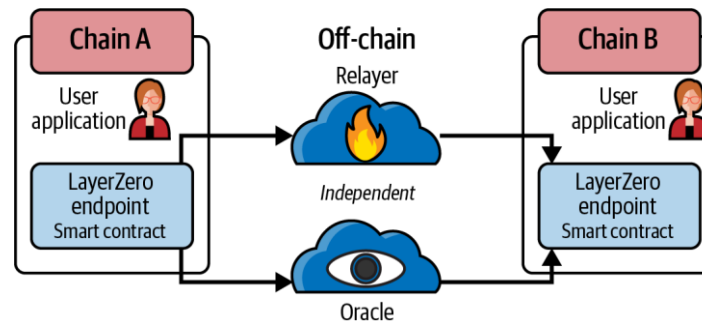


Figure 11-4 — LayerZero cross-chain architecture. Source: Mastering Ethereum 2nd ed. (CC BY-NC-ND).

Real case — Kelp DAO bridge exploit (April 2026)

2026's largest DeFi hack (~\$292M) — and a textbook lesson on why a cross-chain verifier must be decentralized.

What Kelp is

Kelp DAO issues rsETH (restaked ETH). Its token bridges across ~20 chains using LayerZero, whose messages are validated by a DVN (Decentralized Verifier Network).

The weak setup

Kelp ran a 1-of-1 DVN — a SINGLE verifier. LayerZero recommends several independent DVNs; a single one is a single point of failure.

The attack (18 Apr 2026)

Lazarus poisoned the RPC nodes the DVN queried: they compromised two internal nodes and DDoS'd the others to force failover onto the poisoned ones. The DVN then “confirmed” a transfer that never happened, minting 116,500 unbacked rsETH.

Lesson

Same theme as this module: a verifier/oracle with one operator is a single point of failure. Require consensus across independent verifiers. About 25% was later frozen on Arbitrum.

Wormhole — a network of guardians

Originating to bridge Solana ↔ Ethereum (now many chains: BNB, Avalanche, Hyperliquid...). A set of GUARDIANS monitor events on a source chain and sign attestations.

Guardian attestations

Once ENOUGH guardians sign an event, the attestation is valid and the destination chain recognises it (token transfer, governance, NFT...).

Trust trade-off

Combined guardian security reduces single-point-of-failure risk, but requires careful selection and upkeep of the guardian set.

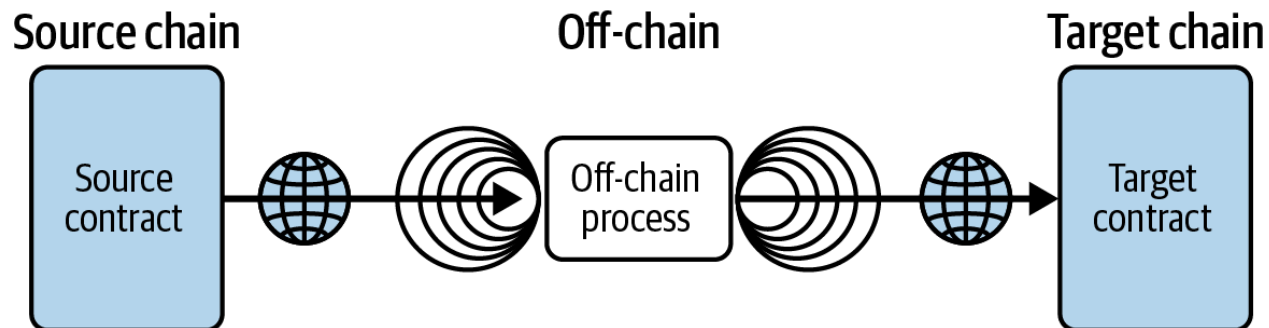


Figure 11-5 — Wormhole cross-chain architecture. Source: Mastering Ethereum 2nd ed. (CCBY-NC-ND).

More interoperability approaches

Chainlink CCIP

Generalized secure messaging + token transfers, built on Chainlink's DON for high trust-minimization. Lock/burn on source → mint/unlock on destination.

Circle CCTP

Native cross-chain transfer of USDC (burn-and-mint). Integrated into Chainlink CCIP.

Cosmos IBC

Built-in interoperability via light clients: each chain stores light clients of the others and verifies their state directly.

Polkadot

Parachains secured by a shared validator set in the Relay Chain, bundling each parachain's transactions into unified consensus.

Each fills a slightly different niche; all carry over the promise — and the trust risks — of oracles, while widening the range of cross-chain use cases.

PART VII

In practice & references

Consuming an oracle in Solidity, best practices, and sources

LAB — DO IT NOW

Validate an oracle before you trust it

Open the folder `lab-oracles/` in Remix (VM). RUN `01-PriceConsumer_naive.sol` to watch a contract trust a stale / negative price; then complete `02-PriceConsumer_start.sol` (positivity + freshness + complete-round checks).

0x4b9C484987F24B51F2b34a14f94F5888387f5fD2

MockV3Aggregator

0x45Cb687BD7e5350644C2F5937aF2cdB419f0ff99

SafePriceConsumer

Deploy MockV3Aggregator(8, 200000000000), poi
SafePriceConsumer(<indirizzo del mock>). Chiama
getSafePrice() sul consumer → ti aspetti
200000000000 (= 2000,00 \$). Poi sul mock chiama
updateAnswer(0) → getSafePrice() reverte
NonPositivePrice; oppure
updateAnswerWithTimestamp(200000000000, 1) →
reverte StalePrice.

In practice: consuming a price feed (Chainlink)

A publish-subscribe feed in Solidity. The contract READS the latest aggregated answer — and validates it before use:

```
import {AggregatorV3Interface} from "@chainlink/.../AggregatorV3Interface.sol";

contract PriceConsumer {
    AggregatorV3Interface internal feed;
    constructor(address feedAddr) { feed = AggregatorV3Interface(feedAddr); } // ETH/USD
    function latestPrice() public view returns (int256) {
        (, int256 answer, , uint256 updatedAt, ) = feed.latestRoundData();
        require(answer > 0, "bad price");
        require(block.timestamp - updatedAt < 3600, "stale price"); // freshness!
        return answer; // USD pairs use 8 decimals
    }
}
```

Best practice: always check the answer is positive AND not stale (updatedAt). Never trust a raw spot price from an AMM for state-changing logic.

Real case — bZx flash-loan oracle attacks (Feb 2020)

The first famous flash-loan attacks. Both abused a price read from a single DEX — exactly what the lab warns against.

What bZx was

bZx (Fulcrum) is a DeFi margin-trading / lending protocol. For prices it relied on Kyber, which sourced from Uniswap — a single, manipulable on-chain price.

The attack (18 Feb)

A flash loan pumped Uniswap's sUSD price to ~\$2 (vs \$1). The attacker deposited sUSD as collateral at that inflated price and borrowed far more ETH than it should have allowed.

Result

Two attacks in one week netted the attackers roughly \$350k then \$650k, while bZx lost ~\$1M — all inside single atomic transactions.

Lesson

Never derive a price from one AMM's spot reserves. Use decentralized feeds / TWAP and validate freshness — the exact defence you built in the lab.

Key takeaways & best practices

- **Oracles are a necessity, not a luxury** — A deterministic chain cannot see the outside world; oracles are the bridge for data, randomness and computation.
- **Model the trust explicitly** — An oracle that can be compromised compromises every contract that trusts it. State your security assumptions.
- **Pick the right pattern** — Immediate-read (one-off), publish-subscribe (frequent feeds), request-response (huge datasets, async).
- **Authenticate the data** — Use authenticity proofs (signatures / ZK, e.g. Chainlink VRF) and/or TEEs — but treat TEEs as not-yet-battle-tested.
- **Prefer decentralized oracles** — DONs + aggregation (median / SchellingCoin) remove the single point of failure and bond nodes economically.
- **Cross-chain widens scope and risk** — LayerZero, Wormhole, CCIP, CCTP, IBC extend reach across ecosystems — with bridge trust assumptions to weigh.

References

- [1] Antonopoulos A., Wood G. et al. — Mastering Ethereum, 2nd ed., Ch. 11 “Oracles” (CC BY-NC-ND). PRIMARY SOURCE. masteringethereum.xyz · github.com/ethereumbook/ethereumbook
- [2] Chainlink documentation — Data Feeds, VRF (Verifiable Random Function), decentralized oracle networks. docs.chain.link
- [3] Chainlink CCIP — Cross-Chain Interoperability Protocol. docs.chain.link/ccip
- [4] Circle CCTP — Cross-Chain Transfer Protocol (USDC). developers.circle.com/stablecoins/ccip
- [5] LayerZero — Omnichain messaging protocol docs. docs.layerzero.network
- [6] Wormhole — cross-chain messaging & guardian network docs. docs.wormhole.com
- [7] Cosmos IBC — Inter-Blockchain Communication protocol. ibcprotocol.org
- [8] Polkadot — parachains & the Relay Chain shared-security model. wiki.polkadot.network
- [9] Lagrange & Brevis — ZK coprocessors for verifiable off-chain computation. lagrange.dev · brevis.network
- [10] EIP-712 typed data; Schelling-point / SchellingCoin concept (V. Buterin, 2014). eips.ethereum.org · blog.ethereum.org

Figure credits & licence

All figures in this deck are reproduced from [Mastering Ethereum, 2nd edition](#) (Antonopoulos, Wood et al.), Chapter 11 “Oracles”.

That work is licensed under [Creative Commons Attribution-NonCommercial-NoDerivatives \(CC BY-NC-ND\)](#). The figures are reused with attribution for non-commercial educational use. Reused figures: 11-1, 11-2, 11-3, 11-4, 11-5.

All explanatory text in this presentation is original course material written for Blockchain & DLT. The Chainlink price-feed snippet follows the standard, publicly documented AggregatorV3Interface pattern.

Questions?

Oracles are the bridge between Ethereum and the real world — powerful, indispensable, and only as trustworthy as the trust model you design around them.

Grazie per l'attenzione



L'Europa investe sul Piemonte, il Piemonte investe su di te